

This value must be string, from docker compose:

```
docker-compose.yml
You, 2 weeks ago | 1 author (You)
1 version: "3.8"
```

In this way we define the services for docker compose, that we want to run:

Note: the *name/s* of services *can be anything*.

```
services:
  frontend:
  backend:
  db:
```

To set the location of *Dockerfile*, for *docker-compose* file, we can use **build**:

```
services:
  frontend:
    build: ./frontend
  backend:
    build: ./backend
  db:
```

To define the image of service, instead of build:

```
db:
  image: mongo:4.0-xeniel
```

To define the env-variables:

```
version: "3.8"

services:
  frontend:
    build: ./frontend
    ports:
      - 3000:3000
  backend:
    build: ./backend
    ports:
      - 3001:3001
    environment:
      - DB_URL=mongodb://db/vidly
  db:
    image: mongo:4.0-xeniel
    ports:
      - 27017:27017
```

Also, we can use object notation for env-variables:

Note: in this way we don't use = we use :

```
environment:
  DB_URL: mongodb://db/vidly
```

To define the volumes (vidly can be any name for volume):

Note: we need to define the name of volume inside the *docker-compose file*

```
version: "3.8"

services:
  frontend:
    build: ./frontend
    ports:
      - 3000:3000
  backend:
    build: ./backend
    ports:
      - 3001:3001
    environment:
      DB_URL: mongodb://db/vidly
  db:
    image: mongo:4.0-xeniel
    ports:
      - 27017:27017
    volumes:
      - vidly:/data/db
```

When we use volumes inside any service of docker-compose.yml file we must declare it, so; we can set volumes outside any service (services and volumes in the same level).

You, 1 second ago | 1 author (You)

version: "3.8"

▷ Run All Services

services:

▷ Run Service

frontend:

build: ./frontend

ports:

- 3000:3000

▷ Run Service

backend:

build: ./backend

ports:

- 3001:3001

environment:

DB_URL: mongodb://db/vidly

▷ Run Service

db:

image: mongo:4.0-xeniel

ports:

- 27017:27017

volumes:

- vidly:/data/db

volumes:

vidly: You, 1 second ago • Uncommitted changes

If we have issues when building images with *docker-compose*, then we can use *--no-cache*

To show the docker compose services that are *relevant to application*: *docker-compose ps*

In *docker-compose.yml-file*, we don't need to define *version-attribute* any more

The default network driver is: *bridge* on Linux and *NAT* on windows.

To list all networks from docker: *docker network ls*

To check the connection between containers:

```
C:\Tests\Docker\Ultimate-Docker-Course-By-Mosh\Tutorial-01\Section-06-Docker-Compose>docker exec -it b35044cf7d13 sh
/app $ ls
Dockerfile      README.md       node_modules    package-lock.json  package.json      public          src
/app $ ping backend
PING backend (172.18.0.4): 56 data bytes
64 bytes from 172.18.0.4: seq=0 ttl=42 time=3.598 ms
64 bytes from 172.18.0.4: seq=1 ttl=42 time=0.217 ms
64 bytes from 172.18.0.4: seq=2 ttl=42 time=0.391 ms
64 bytes from 172.18.0.4: seq=3 ttl=42 time=0.178 ms
64 bytes from 172.18.0.4: seq=4 ttl=42 time=0.163 ms
64 bytes from 172.18.0.4: seq=5 ttl=42 time=0.278 ms
64 bytes from 172.18.0.4: seq=6 ttl=42 time=0.133 ms
64 bytes from 172.18.0.4: seq=7 ttl=42 time=0.180 ms
^C
--- backend ping statistics ---
8 packets transmitted, 8 packets received, 0% packet loss
round-trip min/avg/max = 0.133/0.642/3.598 ms
```

Activate Windows
Go to Settings to activate Windows.

To display the logs of docker-compose services: *docker-compose logs*

```
C:\Tests\Docker\Ultimate-Docker-Course-By-Mosh\Tutorial-01\Section-06-Docker-Compose>docker-compose logs --help
Usage: docker compose logs [OPTIONS] [SERVICE...]

View output from containers

Options:
  --dry-run          Execute command in dry run mode
  -f, --follow        Follow log output
  --index int         index of the container if service has multiple replicas
  --no-color          Produce monochrome output
  --no-log-prefix     Don't print prefix in logs
  --since string      Show logs since timestamp (e.g. 2013-01-02T13:23:37Z) or relative (e.g. 42m for 42 minutes)
  -n, --tail string   Number of lines to show from the end of the logs for each container (default "all")
  -t, --timestamps    Show timestamps
  --until string       Show logs before a timestamp (e.g. 2013-01-02T13:23:37Z) or relative (e.g. 42m for 42 minutes)
```

```
C:\Tests\Docker\Ultimate-Docker-Course-By-Mosh\Tutorial-01\Section-06-Docker-Compose>docker-compose logs backend
backend-1 | > vidly-backend@1.0.0 start /app
backend-1 | > nodemon --ignore './tests' index.js
backend-1 | [nodemon] 2.0.7
backend-1 | [nodemon] to restart at any time, enter `rs`
backend-1 | [nodemon] watching path(s): *.*
backend-1 | [nodemon] watching extensions: js,mjs,json
backend-1 | [nodemon] starting `node index.js`
backend-1 | Server started on port 3001...
backend-1 | Connected to MongoDB: mongodb://db/vidly
backend-1 | Connected to MongoDB: mongodb://db/vidly
```

When building production images, we must *separate the stages* of building using *AS-keyword* in docker file, so we separate the building stage *from production stage*.

When building production app using docker:

- Remove all un-important volumes from *docker-compose*
- Set the *restart-policy* to *on-failure* or *unless-stopped* (the preferred solution is: *unless-stopped*)
- Secure db from un authorized accesses
- *For production Stage* (from previous note) we can use server images to serve the build optimized version of our app (like: nginx-image)
- For running app from *docker-file* we should use *entrypoint* instead of CMD due to *restart-policy*
- When using *custom docker file*, set the *context of file path*. Like . to current path

When using *custom docker file*, and *custom docker-compose file*, for *build-tag* we must define the *context* and *Dockerfile*

```
web:
  build:
    context: ./frontend
    dockerfile: Dockerfile.prod
```

To add tag in docker-compose, we can use image-tag:

Ex → image: jloka:1.0.0

```
image: vidly_api:1
```

The complete *docker-compose file* including the *test service*:

```
services:
  frontend:
    build: ./frontend
    ports:
      - 3000:3000

  backend:
    build: ./backend
    ports:
      - 3001:3001
    environment:
      DB_URL: mongodb://db/vidly
    command: ./docker-entrypoint.sh
  db:
    image: mongo
    ports:
      - 27017:27017
    volumes:
      - vidly:/data/db
  test:
    image: jloka_01-backend
    volumes:
      - ./backend:/app
    command: npm test
    depends_on:
      - backend
    environment:
      DB_URL: mongodb://db/vidly
volumes:
  vidly:
*****
```

The content of docker-entrypoint.sh is:

```
#!/bin/sh
```

```
echo "Waiting for MongoDB to start..."  
./wait-for db:27017
```

```
echo "Migrating the database..."  
npm run db:up
```

```
echo "Starting the server..."
```

```
npm start
```

```
*****
```